

# Neural network decoder

Williamdepig

Note by Williamdepig

November 07, 2025

# Contents

|                                                                        |   |
|------------------------------------------------------------------------|---|
| 1. Neural network decoder for near-term surface-code experiments ..... | 3 |
| 1.1. Synopsis .....                                                    | 3 |
| 1.1.1. Problem .....                                                   | 3 |
| 1.1.2. Solution .....                                                  | 3 |
| 1.1.3. Achievement .....                                               | 3 |
| 1.1.4. Comment .....                                                   | 3 |
| 1.2. Tech details .....                                                | 4 |
| 1.2.1. Tools .....                                                     | 4 |
| 1.2.2. setting for NN .....                                            | 4 |
| 1.3. Training workflow .....                                           | 4 |
| 1.4. Inference workflow .....                                          | 4 |
| 1.5. Engineering Work .....                                            | 5 |
| 1.6. Improvement .....                                                 | 5 |
| 2. Learning high-accuracy error decoding for quantum processors .....  | 6 |
| 2.1. Synopsis .....                                                    | 6 |
| 2.1.1. Solution .....                                                  | 6 |
| 2.1.2. Achievement .....                                               | 6 |
| 2.1.3. Improvement .....                                               | 6 |
| 2.2. Tech details / Inference workflow .....                           | 7 |
| 2.2.1. NN arch .....                                                   | 7 |
| 2.2.2. Input-StablilizerEmbedder .....                                 | 7 |
| 2.2.3. RNN core(pseudo code provided) .....                            | 8 |
| 2.2.4. Readout .....                                                   | 8 |
| 2.3. Training workflow .....                                           | 8 |

# 1. Neural network decoder for near-term surface-code experiments

## 1.1. Synopsis

### 1.1.1. Problem

针对 correlated defects 问题，MWPM 解码器由于 i.i.d error model assumption，导致纠错性能下降。希望使用 neural network 实现比 MWPM 解码器更低的逻辑错误率。

### 1.1.2. Solution

1. Error Model: circuit-level Pauli-noise models; higher or lower probability of applying  $Y$  errors.
2. NN Arch: recurrent NN arch
3. Equipment
  1. Training: DelftBlue supercomputer
  2. Carried: NVIDIA Tesla V100S GPU
  3. CPU: Intel(R) Core(TM) i7-8850H CPU @ 2.60GHz

### 1.1.3. Achievement

1. 使用以上错误模型训练的 NN decoder 在实验数据上的测试性能( $\epsilon_L = 3.0\%$  on  $d = 3$ )超过了 MWPM 解码器( $\epsilon_L = 4.5\%$  on  $d = 3$ )，与 maximum-likelihood decoding(TN)相当。
2. 对于实际 qpu 产生的 syndromes，NN 不需要依赖对其错误概率、缺陷和纠错方法进行建模（如 TN 需将其编码为超图结构）。

### 1.1.4. Comment

1. 使用实验数据如何训练，如何设置 loss func 和 backpropagation?
  1. 实验数据的 output 为，在经历  $n$  轮纠错循环后，在最后一轮进行破坏性的测量，看结果是否发生逻辑反转，事实上做不了模拟中实现的 recurrent NN。
2. 可扩展性老生常谈，如何处理当码距增加时的训练难度指数级增加问题？
3. 声称不需要任何对 error models 的先验知识，这是伪命题。
  1. 使用模拟数据进行训练本身就需要设定 error model
  2. 使用实验数据来训练时，实际上不也是在建模 error model 吗，只不过像黑箱一样隐藏在神经网络的内部参数中了
  3. 显然使用实验数据训练得到的 NN 不能够应用在其他 qpu 上，只能专用于这一个 qpu，并且一旦其物理性质发生了一些改变，也需要重新训练、
4. 预测的本质：解码器的本质是在每一轮忠实地记录根据 syndrome 解码出来的比特可能的错误情况，并判断这些错误是否构成一个逻辑错误，并在经典计算机上实时保存

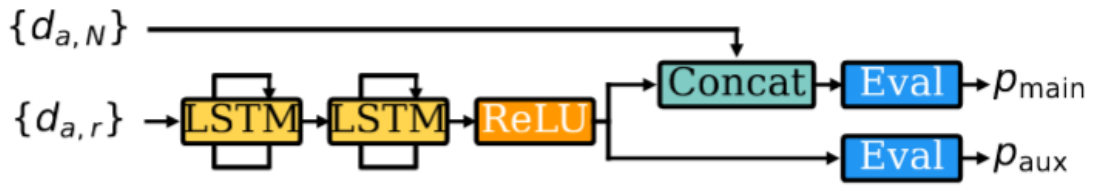
当前逻辑比特状态，并不会直接对量子比特进行纠错（这反而可能引入新的错误，也极大地耗费时间降低性能），使用表面码等拓扑码就是为了不需要实时纠错不是吗。

## 1.2. Tech details

### 1.2.1. Tools

1. stim
2. TensorFlow
3. Adam optimizer

### 1.2.2. setting for NN



1. **Input:**  $d_{a,r}$  是模拟中的每轮 syndrome,  $d_{a,N}$  是最后纠错循环结束后对数据比特测量结果推断出的 syndrome
2. **network:** 两层 LSTM 和一个 ReLU 激活层, 后面跟着评估层, 有两个接收头, 一个仅接受  $d_{a,r}$  并作出对逻辑错误率的预测, 一个将  $d_{a,r}$  与  $d_{a,N}$  整合后做出预测
3. **output:** 转换为二分类问题: “是否发生逻辑错误?” 预测纠错循环结束后的逻辑错误率
4. **cost function:**  $I = H(p_{\text{main}}, p_{\text{true}}) + w_a H(p_{\text{aux}}, p_{\text{true}})$

## 1.3. Training workflow

1. data generation:
  - error model setting
  - data format: `([d_a, r=1...N-1], [d_a, N], p_true)`
  - (soft information)
2. model training:
  - `d_a, r=1...N-1` → `recurrent_input`
  - `d_a, N` → `final_input`
  - `p_true` → `p_main, p_aux`

## 1.4. Inference workflow

1. input: `d_a, r=1...N`
2. output: `p_main`
3. logical qubit measuring result: `M_L`
  - `p_main > 0.5` → `M_L ^ 1`

- `p_main<=0.5`  $\rightarrow$  `M_L`

## 1.5. Engineering Work

1. **Stim** generates `(defects, label)` data pair
2. Simple neural network construction(**TensorFlow**)
3. Loss function setting(**keras**)
4. **MWPM**(`pymatching`) for benchmark

## 1.6. Improvement

1. 本文中的 NN 仅使用模拟数据进行训练，并未使用实验数据
2. NN 的可扩展性是一个问题，本文使用的 **recurrent** NN 不具有可扩展性，

## **2. Learning high-accuracy error decoding for quantum processors**

### **2.1. Synopsis**

AlphaQubit

#### **2.1.1. Solution**

1. Error Model: richer noise model than those considered in previous machine-learning decoder work—including leakage, cross-talk and soft readouts with an amplitude damping component
2. NN Arch: recurrent-transformer-based neural-network architecture that learns to predict errors in the logical observable based on the syndrome inputs

#### **2.1.2. Achievement**

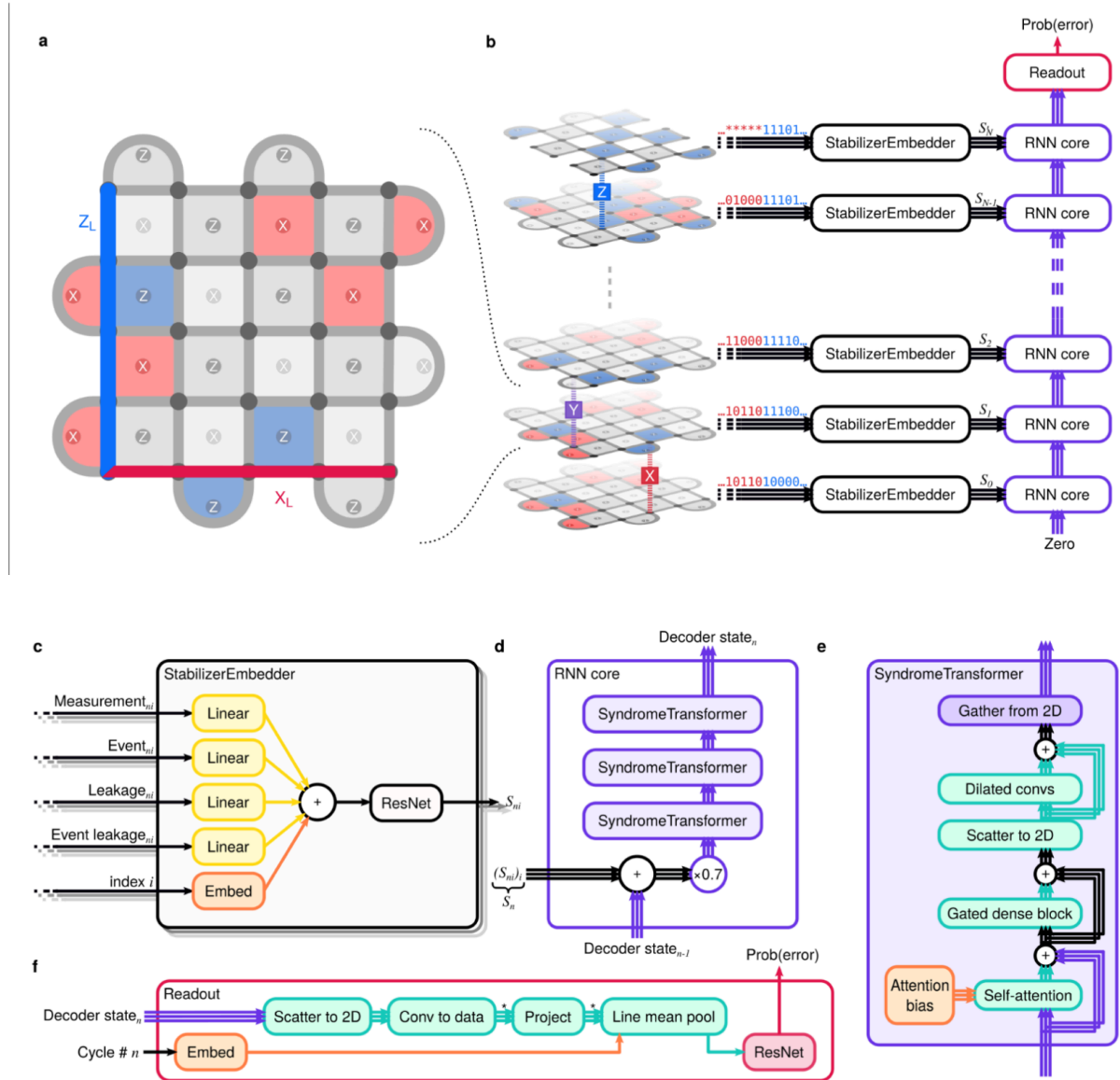
1. 优于张量网络
2. 规模增加（码距增加至 11）时精度依然维持，优于 soft-input-augmented correlated matching method.
3. 能泛化至远超训练范围的轮次
4. 本解码器的核心优势在于从真实实验数据中学习的能力。这使得它能够直接利用表征 I/Q 噪声和泄漏效应的丰富输入，无需为每个特征手动设计特定算法。(?)

#### **2.1.3. Improvement**

1. 码距增加到 11 后的训练难度和训练数据需求量都显著增加
2. 需要改进解码器吞吐量

## 2.2. Tech details / Inference workflow

### 2.2.1. NN arch



The decoder architecture is implemented and trained using the JAX, Haiku and JAXline ML frameworks.

### 2.2.2. Input-StablilizerEmbedder

将多模态输入转换为统一的 representation

1. 提供 1-4 个输入，传入各自的 Linear 层

- Measurement: 稳定子测量结果

$$\text{post}_1 = \text{the posterior probability of observing a } |1\rangle \text{ state} \quad (1)$$

- Event: 即 defect, 检测产生翻转的事件

- \*Leakage: 泄漏概率

$$\text{post}_2 = \text{the probability of having seen leakage} \quad (2)$$

- \*Event leakage: temporal-difference analogue

2. 将 index  $i$  传入 Embed 层, 以提供对物理布局的感知
3. 将第一步特征向量和第二步位置向量累加融合为一个稳定子的表征向量
4. 将每个表征提供给 2-layer ResNet, 生成提供给 RNN 的表征  $S_{ni}$
5. 末轮需做特殊处理: As the final-round stabilizers are not measured but computed from the data qubits, we encode this by using a separate embedding for the final round, with a separate final-round linear projection for the on-basis computed stabilizers and a single learned embedding for all the undefined off-basis stabilizers (where on- and off-basis stabilizers are defined in ‘The rotated surface code’).

### 2.2.3. RNN core(pseudo code provided)

依序列捕捉空间局域性质和长程非局域性质

1. 合并上一次解码器状态  $X_{n-1}$  与本次稳定子表征  $S_n$
2. 循环多次输入给 SyndromeTransformer:
  1. layer\_norm
  2. Multi-head Attention with bias: 捕捉长程、非局域关联, 即任意稳定子间的关联, bias 告诉模型哪些稳定子对更值得关注
  3. layer\_norm
  4. GatedDenseBlock: FFN
  5. ScatteringResidualConvBlock: 捕捉局部特征
    - Scatter to 2D: 将稳定子向量列表依其物理上的排列铺到 2D 网格上
    - Dilated convs
    - Gather from 2D
3. 输出解码器状态  $X_n$

### 2.2.4. Readout

1. Scatter to 2D  $\rightarrow$  Conv to data: 将稳定子状态转化为 2D 网格上的数据比特状态
2. Project: 特征降维 ( $1 \times 1$  卷积?)
3. Line mean pool: 沿与 logical observable 垂直的行或列进行平均池化
4. Cycle embedding: 提供实验轮次信息
5. ResNet and output: 给出最终 Prob(error)

## 2.3. Training workflow

1. **Pretraining:** 用 Stim 生成大量基础 memory experiment 的数据, 让模型学习基本解码规则, 使用 DEMs、XEB、SI1000 等不同 error model 先后训练。



2. **Finetuning**: 使用来自 Sycamore 处理器的实验数据或者 Pauli+ simulator 的数据（模拟了 leakage, cross-talk）进行调优。
3. **Loss func**: 交叉熵。随机梯度下降优化。